

8.4.1 消息队列的结构

消息队列在内核中的表示是队列数据结构，如图 8.8 所示。当创建新的消息队列时，内核将从系统内存中分配一个队列数据结构，作为消息队列的内核对象。可以看到，这个对象有其对应的权限，以及消息头部指针。队列的消息由这个头部指针引出，每个消息都有指向下一个消息的指针（或者为空）。这是一种常见的队列的链表设计。在消息的结构体中，除了“下一个”指针之外，就是消息的内容。消息的内容包含两部分：数据和类型。数据是一段内存数据，和管道中的字节流相似。类型是用户态程序为每个消息指定的。在消息队列的设计中，内核不需要知道类型的语义，仅仅保存并基于类型进行简单的查找。如图 8.8 所示，第一个消息的类型是 1。这可能代表消息中的数据是一个字符串，也可能代表该数据是一个结构体。类型的具体意义需要用户态程序自己来管理。

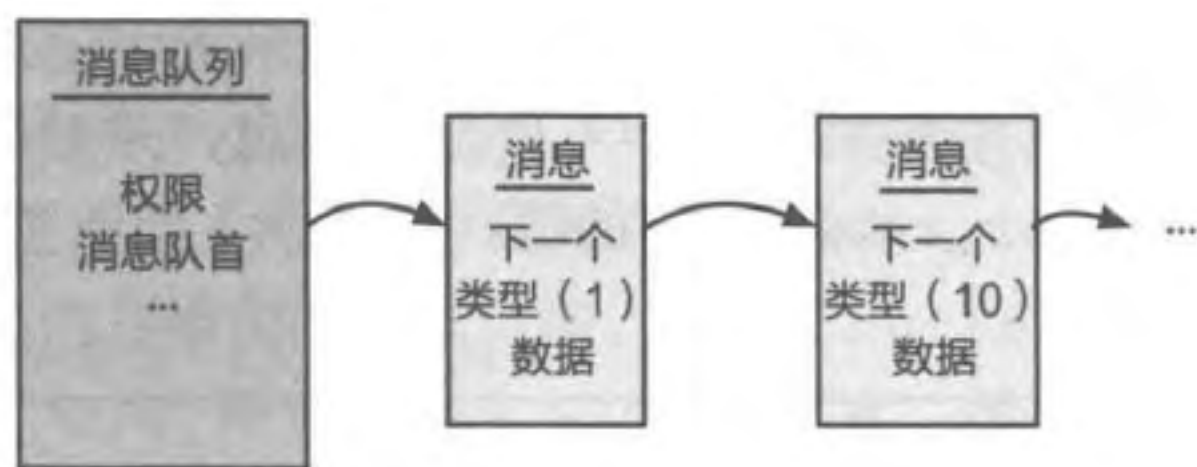


图 8.8 消息队列

8.4.2 基本操作

消息队列的操作一般被抽象为四个基本操作：`msgget`、`msgsnd`、`msgrcv`、`msgctl`。这四个操作在 Linux 系统上被实现为系统调用。`msgget` 允许进程获取已有消息队列的连接，或者创建一个新的消息队列。消息队列本质上采用的是信箱的通信方式。发送者和接收者在通信过程中，只需要建立好对同一个信箱（此处的“队列”）的访问，就相当于建立了通信连接。只要有对应的权限，消息队列允许任意数量的进程连接到同一通信连接上（即同一队列上）。`msgctl` 可以控制和管理消息队列，如修改消息队列的权限信息或删除该消息队列。

进程可以通过 `msgsnd` 向消息队列发送消息，通过 `msgrcv` 从消息队列接收消息。多个进程可以同时向队列发送消息或从队列接收消息。消息的发送和接收成功的标志，是消息被放到队列上或者从队列上取出。大部分情况下这两个过程是非阻塞的：对发送者来说，只要队列有空闲的空间就可以向队列发送消息，而接收者只要有未读消息就可以直接读取消息并完成操作。若发送消息时消息队列没有可用空间或接收消息时没有未读消息，默认的操作是阻塞进程，直到有空间腾出或者新的消息到来。Linux 内核在发送和接收消息的接口（`msgsnd` 和 `msgrcv`）上允许用户指定是否等待（`NOWAIT` 选项）。若用户指定了 `NOWAIT`，内核可以直接返回相关的错误信息，用户进程不会阻塞。

练习

8.8 消息队列是否支持类似超时机制的特性？请解释理由。

8.5 案例分析：L4 微内核的 IPC 优化

本节主要知识点

- L4 针对进程间通信展开了哪些优化？
- L4 的数据传递（或消息传递）机制是怎样设计的？

早期微内核 Mach 的复杂性对进程间通信的性能影响很大。根据 Mach 的经验，Liedtke 等研究人员开始研发 L4 系列的微内核系统^[1-4]。L4 系列微内核系统设计的一个突出思路是：进程间通信是微内核的核心功能，需要围绕通信完成整个系统的设计和实现。L4 是当下仍然十分主流的微内核系统，特别是后续衍生出了各种变体和相关的系统。图 8.9 给出了 L4 系统从 1993 年开始的一些发展脉络^[8]。本节内容将以 L4 早期 IPC 设计为主，同时介绍相关设计在后续系统中的使用和优化。

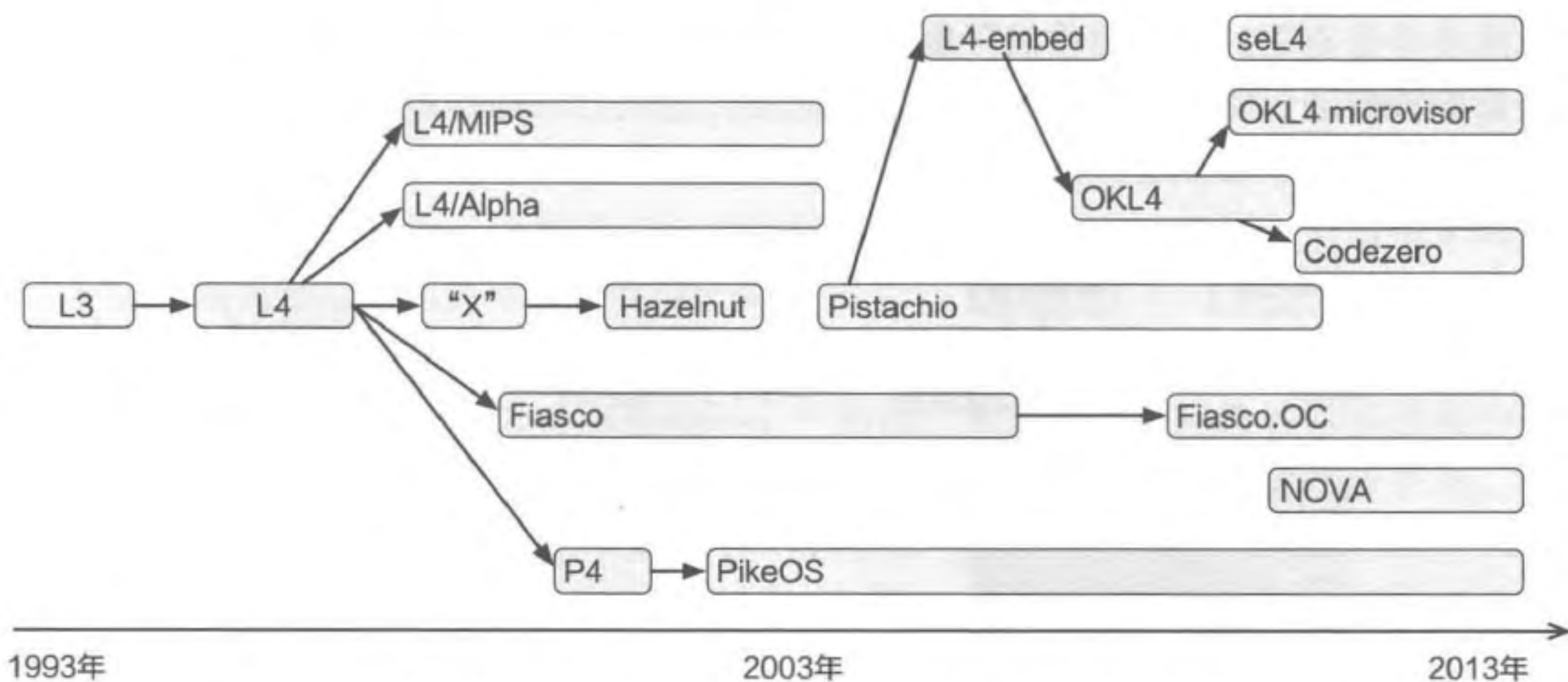


图 8.9 L4 微内核家族及其演进，简化自研究者 Gernot Heiser 的总结^[8]

在 L4 微内核中，内核只保留了基本的功能，包括地址空间、线程、进程间通信等，并且不考虑兼容性等要求，而是选择针对特定硬件做极致的性能优化。这样做的好处是内核的代码量非常少，可以为少量的功能提供尽可能完善的支持。下面将从消息传递^①、控制流转移、通信连接和权限检查四个方面介绍 L4 中 IPC 的设计。

8.5.1 L4 消息传递

在 L4 早期的设计中，为了尽可能减少内核暴露的接口，给单个通信接口增加了丰富的语义。这一点主要体现在传递的消息上：除了类似函数调用的消息外，还支持几乎

① 由于 L4 的设计中均使用消息接口，因此我们直接使用“消息传递”来替代“数据传递”。

任意大小和类型的信息，如对齐的数据缓冲区或者字符串等。此外，L4 的一些系统（如 seL4^[6]）还支持以消息的方式传递 Capability。

抛开上层接口的细节，对于内核来说，L4 根据要传递的消息大小，将其分为短消息和长消息，并且设计了不同的传输方式。在后续基于 L4 扩展的微内核系统中，也有中等长度消息的概念以及相关的优化设计。不管是哪种设计，L4 都在尽可能减少数据拷贝，从而优化通信过程中的开销。下面分别介绍短消息和长消息的设计。

短消息

当传递的消息较短时，L4 会直接使用寄存器的参数传递方式来实现零拷贝传输，如图 8.10a 所示。具体来说，在调用的接口上，发送者进程将参数放置在寄存器上。下陷到内核后，内核可以直接从发送者的上下文切换到接收者的上下文，并且不会修改存放在寄存器中的参数。这种方式可以在不触摸数据的情况下完成消息的传递。通过寄存器来传递跨进程的参数存在的一个缺陷是，能够传递的数据量依赖于具体的硬件架构。如在 x86-32 这样的 32 位硬件上，能够使用寄存器（通常是通用寄存器的一部分）传递的参数大小十分有限。除了硬件的限制外，内核和用户态之间的交互接口（系统调用 ABI）也会影响能够通过寄存器传递的数据量。此外，这也会增加移植系统到新硬件或者是支持新的 ABI 接口的复杂性。

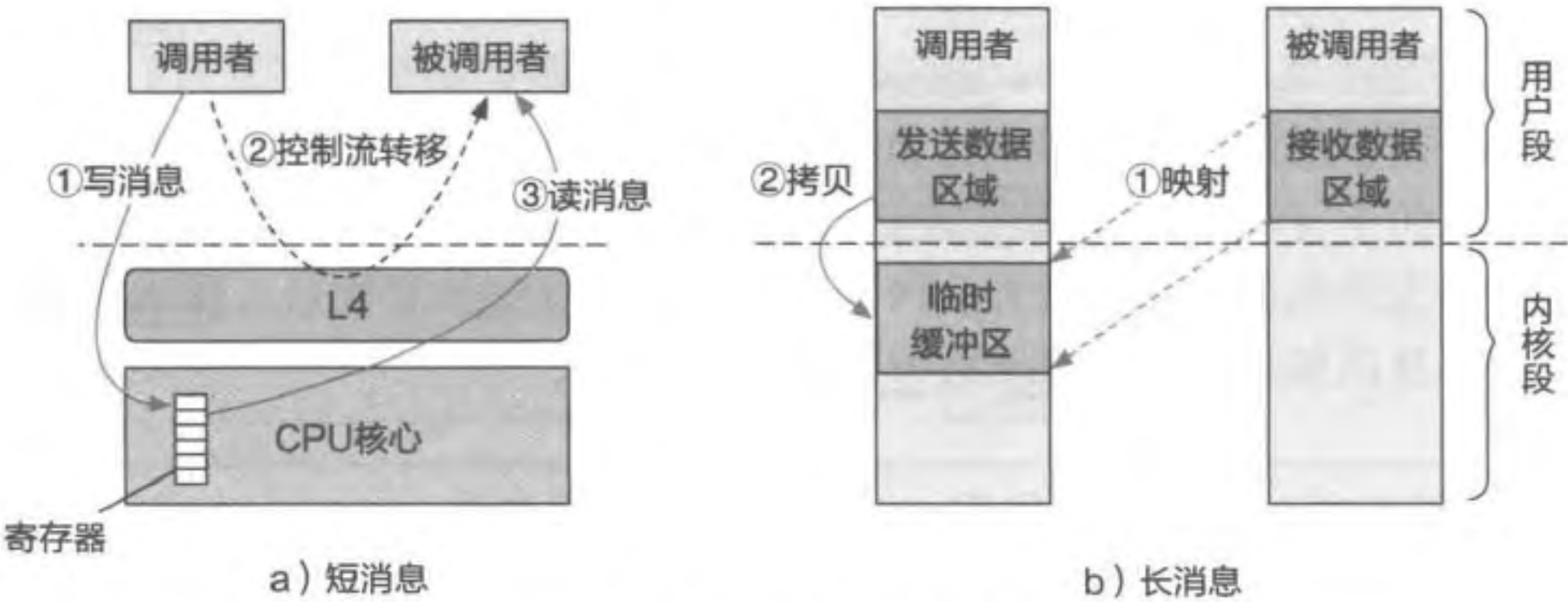


图 8.10 L4 短消息和长消息的传输方式

Pistachio (L4 系列中的一个变体) 引入了虚拟消息寄存器 (Virtual Message Register) 的概念，允许用户态自定义虚拟消息寄存器集合的大小（如 64 字节）。这里的思想是将“寄存器参数”传递和具体的硬件寄存器解耦，使用虚拟的寄存器来实现功能。内核会将其中一些虚拟消息寄存器映射到物理寄存器，而物理寄存器放置不下的部分则包含在每个线程固定地址的内存空间中（相当于通过内存虚拟了一些额外的寄存器）。通过在微内核的用户态软件层封装寄存器使用的接口，可以对应用隐藏物理寄存器和内存虚拟的寄存器之间的区别。由于内存中的虚拟消息寄存器不会很多，传输的开销（虚拟消息寄存器需要拷贝）仍然在可控的范围内。后续的微内核系统，如 seL4 和 Fiasco.OC 沿用了用

寄存器传递小数据和虚拟消息寄存器的方法。

长消息

L4 长消息的传输本质上是由内核辅助的，通常需要两次拷贝：发送者用户态拷贝到内核缓冲区，以及内核缓冲区拷贝到接收者用户态。L4 对长消息的传输做了不少优化，此处介绍两点。

- 第一点优化是拷贝次数的优化。在 L4 中，内核通过建立一个临时的映射区域来传输数据，可以实现一次拷贝的数据传输，如图 8.10b 所示。具体来说，L4 在每个进程的内存地址空间中预留一段区域，用作建立临时缓冲区。这段区域在不同的进程中是不共享的（即使它处于内核段）。当发送者发起消息传输时，内核在发送者的上下文中执行，并将发送者临时缓冲区的虚拟空间映射到接收者接收消息的物理内存区域上。完成这次映射后，内核可以直接将消息从发送者的用户态内存区域拷贝到临时缓冲区，从而完成将数据从发送者拷贝给接收者的过程。
- 第二点优化体现在接口层面上。消息传递的接口通常要求传递的数据落在一段连续的虚拟地址区域，如果消息落在不同区域中，通常只能通过多次通信来将它们分别发送出去。在 L4 中，长消息可以在单个通信调用中指定多个缓冲区。由于最终的数据是由内核拷贝到临时映射区域来完成传输的，接收不连续的消息不会引入额外的传输开销。这种接口上的优化事实上分摊了多次进程间通信带来的上下文切换和特权级切换导致的性能开销。

长消息对于 POSIX 这类原本就依赖缓冲区域进行数据传递的接口有较好的兼容性。不过，在实际使用中，长消息机制会遇到不少问题。例如，在长消息的拷贝过程中可能出现缺页异常。这既可能发生在发送者地址空间，也可能发生在接收者地址空间。由于拷贝本身是在内核态进行的，这就需要内核回到用户态的页管理程序去处理异常（L4 系列的很多系统是将页面错误处理程序放在用户态的）。这种内核操作依赖于用户态的处理显著加剧了内核的复杂性，比如用户态页处理程序完成处理后，内核需要重新建立原始的系统调用上下文，再执行通信操作。后续的 L4 系统逐渐倾向于直接在通信双方的进程间建立共享内存区域，由进程自己负责大量数据的传输，从而简化内核的通信逻辑。

8.5.2 L4 控制流转移

L4 控制流转移的整体过程^①和前文介绍的过程（8.1.4 节）是类似的。本节介绍 L4 中引入的两个优化控制流转移性能的机制：惰性调度（Lazy Scheduling）以及直接进程切换（Direct Process Switching）。

惰性调度

通信的控制流转移往往是通过内核的线程 / 进程管理和调度来实现的。在 L4 同步的

^① 这里只关注 L4 的同步双向 IPC。

IPC 模型下，线程的状态经常在就绪和阻塞中交替（发送消息后进入阻塞状态）。这意味着在通信的过程中会发生频繁的调度队列操作：一个线程会在短时间内多次被移入和移出就绪调度队列。这些额外执行的代码和访问的数据会在通信过程中引入 TLB 不命中、缓存不命中等开销。

为了解决这个问题，L4 提出了惰性调度的优化方式，如代码片段 8.14 所示。当线程在 IPC 操作上阻塞时，内核会在线程管理结构（Thread Control Block, TCB）中更新其状态，但会将线程保留在就绪队列中。调度器在调度线程时会遍历就绪队列，忽略这些处于阻塞状态的线程，直到找到真正可运行的线程。这样的设计可以避免大量的队列操作：IPC 过程中即使线程状态发生变化，也只需要修改 TCB 中的结构，而不需要调度队列的操作。不过，这也潜在地增大了调度器的复杂性和性能开销。需要注意的是，这种方案是基于一个假设的：IPC 相关线程的阻塞状态会很快结束（比如发送者接收到返回消息后）。

代码片段 8.14 惰性调度示例伪代码

```
1 // 惰性调度下，不会访问调度队列，而是配置进程结构体的 status 状态
2 int lazy_schedule(TCB)
3 {
4     Cur->status = IPC_blocked;
5     TCB->status = Running;
6     ...
7 }
8
9 // 正常调度下，将当前线程放在等待队列，将目标线程放在可运行队列
10 int normal_schedule(TCB)
11 {
12     Move_cur_to_block_list();
13     Move_TCB_to_runnable_list();
14     ...
15 }
```

惰性调度也存在自己的问题。惰性调度优化导致调度器的就绪队列中可能存在大量的阻塞线程，从而使调度器的执行时间与系统的线程数及其 IPC 执行情况相关。这使其很难被应用在对实时性有较高要求的场景中。研究者已经提出了一些解决方案^[9]，感兴趣的读者可以深入研究。

直接进程切换

早期的微内核系统允许内核在 IPC 控制流转移过程中执行调度程序，而调度的不确定性会严重影响 IPC 的时延。因此，L4 将调度程序从控制流转移的关键路径上移除，即直接完成从调用者到被调用者的切换（返回消息的过程类似）。这种方法称为直接进程切换。直接进程切换除了能加速控制流的转移外，还有额外收益。一个主要的例子是缓存。避免了控制流转移中其他进程的干预以及调度的影响后，对于调用者发来的数据，被调

用者可以以缓存命中的方式直接访问。这对于加速被调用者处理请求是有“隐式”提升的。

不过直接进程切换也有一些问题，如可能破坏实时场景下任务的优先级。如果只要调用者发起通信，被调用者就一定响应，那么在一些实时的场景下将无法保证对不同优先级任务的区分处理。L4 的后续系统会在进程间通信的过程中，通过比较调用者和被调用者的优先级来判断是否能够直接调用。

8.5.3 L4 通信连接

L4 系列的微内核经历了直接通信到间接通信的转变。早期 L4 的设计使用线程作为通信的目标，这是为了避免过多的中间抽象带来的缓存和 TLB 污染的性能开销。Liedtke 曾经指出，用类似 Mach 中的端口的抽象，可能会产生 12% 的开销^[2]。

不过，线程作为通信目标的方案也有一些问题^[8]。首先，该模型要求线程 ID 是全局唯一的标识符。全局 ID 在后续的系统工作中被证明会引入潜在的隐蔽信道（Covert Channel）的危险，而这会导致攻击和信息泄露的发生。其次，该模型的信息隐藏性差。多线程服务必须向客户端进程公开其内部结构，比如包含几个线程，每个线程的 ID 是多少，不然就很难进行负载均衡相关的操作。随着现代处理器的能力变得越来越强，并且支持大页等机制，间接通信带来的 TLB 污染问题已经缓和了很多。

这些改变使得后续的系统更倾向于基于信箱的间接通信。例如 seL4 和 Fiasco.OC 最终仍然使用类似 Mach 的方案，将端口作为通信的目标。

8.5.4 L4 通信控制（权限检查）

L4 经历了直接通信到间接通信的转变，其通信控制（或者说权限检查）在这两类通信连接方式下的设计也有所不同。

直接通信下，只要有对应的标识符（如进程号），一个进程就可以尝试和另一个进程通信。在 L4 的早期设计中，内核在通信过程中会将发送者的一个“不可伪造的标识符”传递给接收者，帮助接收者判断是否响应发送者的消息。然而，恶意的发送者进程可能用大量的消息来“轰炸”接收者进程。接收者进程需要花费大量的时间来检查并丢弃恶意消息，这也会影响其执行有用的工作。这样的“轰炸”其实构成了拒绝服务的攻击。

早期 L4 通过氏族和酋长（Clans and Chiefs）^[10]的机制来解决这个问题。具体来说，整个系统内的进程按照“氏族”的层次结构进行组织，每个氏族都有一个指定的“酋长”。在氏族内部，所有消息都是自由传输的。而跨氏族的消息（无论是传出的还是传入的）都将重定向到氏族酋长，由其来控制消息的流向。氏族和酋长机制除了保护内部的进程外，还能够限制不受信任的进程。即如果存在某个不受信任的进程希望将在内部获取的敏感信息传递到外部进程，那么会被酋长检测到并阻止。

氏族和酋长在后续的 L4 系统中逐渐被放弃^[8]。这是因为其有以下问题。首先，一

旦消息需要发到外部，那么中间会经过几次重定向，通过一层层的酋长往外发送。每次这样的重定向相当于增加了两次 IPC 的调用，而这显然是相当大的开销。其次，即使不考虑性能开销，酋长本身仍然是可能被攻击的一个点。恶意的攻击者可以通过攻击酋长来限制整个氏族和外部的通信。

在间接通信的场景下，L4 系列的微内核组件采用更灵活的权限机制 Capability 来解决上述问题。简单来看，Capability 是对于内核对象（可以将微内核中几乎所有的内核结构都抽象成内核对象，包括 IPC、内存等）的索引，以及对于该内核对象的权限。以 IPC 为例，微内核会在内核中为每个通信连接维护一个 IPC 内核对象。这个内核对象中包含接收者、发送者、缓冲区等和通信相关的信息。发送者进程要发起通信时，需要告知内核一个特定的 Capability 来发起通信，内核负责检查 Capability 的正确性及其权限，然后通过 Capability 找到对应的内核对象以及与之对应的接收者，从而开始通信过程。这也就意味着，两个进程间要通信，必须首先通过命名服务等方式获取对应的 Capability，否则它们会在执行通信逻辑时被内核拦截下来，从而避免前文提到的拒绝服务的攻击。

在现代的主流微内核系统中，几乎都会使用 Capability 来管理内核对象并负责通信的权限检查。

练习

8.9 L4 的直接进程切换主要避免了哪部分通信开销？

8.6 案例分析：LRPC 的迁移线程模型

本节主要知识点

- L4 中的直接进程切换和 LRPC 中的迁移线程模型的异同分别是什么？
- LRPC 的数据传递（或消息传递）机制是怎样设计的？

这一节将介绍一种比较“极端”的优化性能的 IPC 设计：迁移线程（Thread Migration）。截至目前，我们了解到优化 IPC 性能的大部分工作关注两个部分：优化控制流切换的性能和优化数据传输的性能。迁移线程认为，可以将其他的 IPC 设计看成将需要处理的数据发送到另一个进程处理，这也是为什么控制流切换和数据传输会成为主要的瓶颈。如果换一个角度，将另一个进程处理数据的代码拉到当前进程，那么我们是不是可以避免控制流的切换（仍然是当前进程处理）以及数据传输（数据已经准备在当前进程中）？迁移线程就是围绕这个新的视角进行设计的。